Creation of tables (SQL)

The tables were created in a specific order. Independent tables such as EventType, Room, Staff, and Member were created first because they do not rely on other tables. Dependent tables such as Event were created afterwards, as they contain foreign keys. Finally, EventBooking, RoomPayment, and Feedback were created last, as they depend on the existence of records in related tables.

The query in Figure 7., creates Room table. RoomID is set as the primary. INT is used for numbers, VARCHAR for text fields. The Hourly_Rate uses DECIMAL (6,2) for accurate currency values, adjusted from DECIMAL (3,2) in the scenario.

```
CREATE TABLE Room (
  RoomID INT PRIMARY KEY,
  Room_Name VARCHAR(50),
  Room_Capacity INT,
  Hourly_Rate DECIMAL(6,2),
  Facilities VARCHAR(200)
);
```

Figure 7. Creating “Room”.

The query in Figure 8., creates table, EventType. EventTypeID is primary key. VARCHAR (100) allows sufficient number of characters for event type names.

```
CREATE TABLE EventType (
   EventTypeID INT PRIMARY KEY,
   Description VARCHAR(100)
);
```

Figure 8. Creating “EventType”.

The query in Figure 9., creates a table Member. MemberID is the primary key with AUTO_INCREMENT, starting from 100, so each member is automatically given a unique ID. VARCHAR is for text fields allowing input of different lengths, Member_Lname was adjusted from 5 in the scenario to 50 allowing surnames to be longer than 5 characters.

```
CREATE TABLE Member (
   MemberID INT AUTO_INCREMENT PRIMARY KEY,
   Member_Fname VARCHAR(50),
   Member_Lname VARCHAR(50),
   Member_Email VARCHAR(50),
   Member_Phone VARCHAR(30),
   Member_Type VARCHAR(30)
) AUTO_INCREMENT = 100;
```

Figure 9. Creating “Member”.

The query in Figure 10., creates table Staff. The StaffID is the primary key, and VARCHAR is used for text fields like name and email.

```
CREATE TABLE Staff (
  StaffID INT PRIMARY KEY,
  Staff_Name VARCHAR(100),
  Staff_Email VARCHAR(50)
);
```

Figure 10. Creating “Staff”.

The query in Figure 11., creates an Event table. The EventID is primary key, and Event_Duration includes a CHECK constraint to ensure it is greater than 0. The table uses foreign keys to link to EventType, Room, Staff, and Member. The Ticket_Cost field uses DECIMAL (6,2), which was adjusted from the scenario DECIMAL (3,2) to allow more accurate values.

```
CREATE TABLE Event (
  EventID INT PRIMARY KEY,
  Event_Name VARCHAR(100),
 EventTypeID INT NOT NULL,
  Event_Date DATE,
  Event_Duration INT CHECK (Event_Duration > 0),
  RoomID INT NOT NULL,
  StaffID INT NOT NULL,
```

```
  MemberID INT NOT NULL,
  Ticket_Cost DECIMAL(6,2),
  FOREIGN KEY (EventTypeID) REFERENCES EventType(EventTypeID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
  FOREIGN KEY (StaffID) REFERENCES Staff(StaffID),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);
```

Figure 11. Creating “Event”.

The statement in Figure 12., creates a table EventBooking, BookingID is primary key. The table defines foreign key constraints: MemberID references the Member table and EventID references the Event table.

```
CREATE TABLE EventBooking (
  BookingID INT PRIMARY KEY,
  MemberID INT NOT NULL,
  EventID INT NOT NULL,
  Booking_Date DATE,
  Booking_Status VARCHAR(50),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
```

Figure 12. Creating “EventBooking”.

The code in Figure 13., creates a Feedback table. FeedbackID is an auto-incrementing primary key that uniquely identifies each record. EventID is a foreign key referencing Event table. Feedback_Date is a DATE field with a default value of CURRENT_DATE, automatically recording the submission date if none. Feedback_Rating is constrained by a CHECK condition

```
CREATE TABLE Feedback (
  FeedbackID INT AUTO_INCREMENT PRIMARY KEY,
  EventID INT NOT NULL,
  Feedback_Date DATE DEFAULT CURRENT_DATE,
  Feedback_Rating INT CHECK (Feedback_Rating > 0 AND Feedback_Rating < 6),
  Feedback_Comments TEXT,
  FOREIGN KEY (EventID) REFERENCES Event(EventID)
) AUTO_INCREMENT = 500;to values between 1 and 5.
```

Figure 13. Creating “Feedback”.

The code in Figure 14., creates RoomPayment table. PaymentID is an auto-incremented primary key that uniquely for each payment. The Amount_Paid DECIMAL (3,2) from the scenario was adjusted to 6,2 for accurate input. Foreign key constraints ensure each payment references an existing event and member.

```
CREATE TABLE RoomPayment (
```

```
    PaymentID INT AUTO_INCREMENT PRIMARY KEY,
    EventID INT NOT NULL,
    MemberID INT NOT NULL,
    Payment_Date DATE,
    Amount_Paid DECIMAL(6,2),
    Payment_Method VARCHAR(30),
    Payment_Status VARCHAR(30),
    FOREIGN KEY (EventID) REFERENCES Event(EventID),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
) AUTO_INCREMENT = 200;
```

Figure 14. Creating “RoomPayment”.